Electronics & Computing Club

Mini Synth Project

Build & Code: Arduino Workshop

Written by Charles Ross on January 31st, 2026

Section 1
Overview

Time for some music! Arduinos are fast devices, capable of changing their pins from 1s to 0s at incredible speeds. We can take advantage of this to generate sounds and play music from it!

To accomplish this, we need to understand what sound is. When dealing with audio in electronics, we can think of sound as a sinusoidal voltage wave, going up and down in a cycle. When we send that wave to a speaker, the speaker will vibrate at the same movement as the wave, making small changes in air pressure that we can hear as sound.

To do this on the Arduino, we can take advantage of the board's quick-switching pins to go from 1, to 0, to 1 again, essentially creating a voltage wave. Then, we can control the speed of that switching, generating different tones and making a full instrument! Sure, we might be missing some of the bells and whistles of a true synthesizer, but this project is a great start if audio electronics interest you!

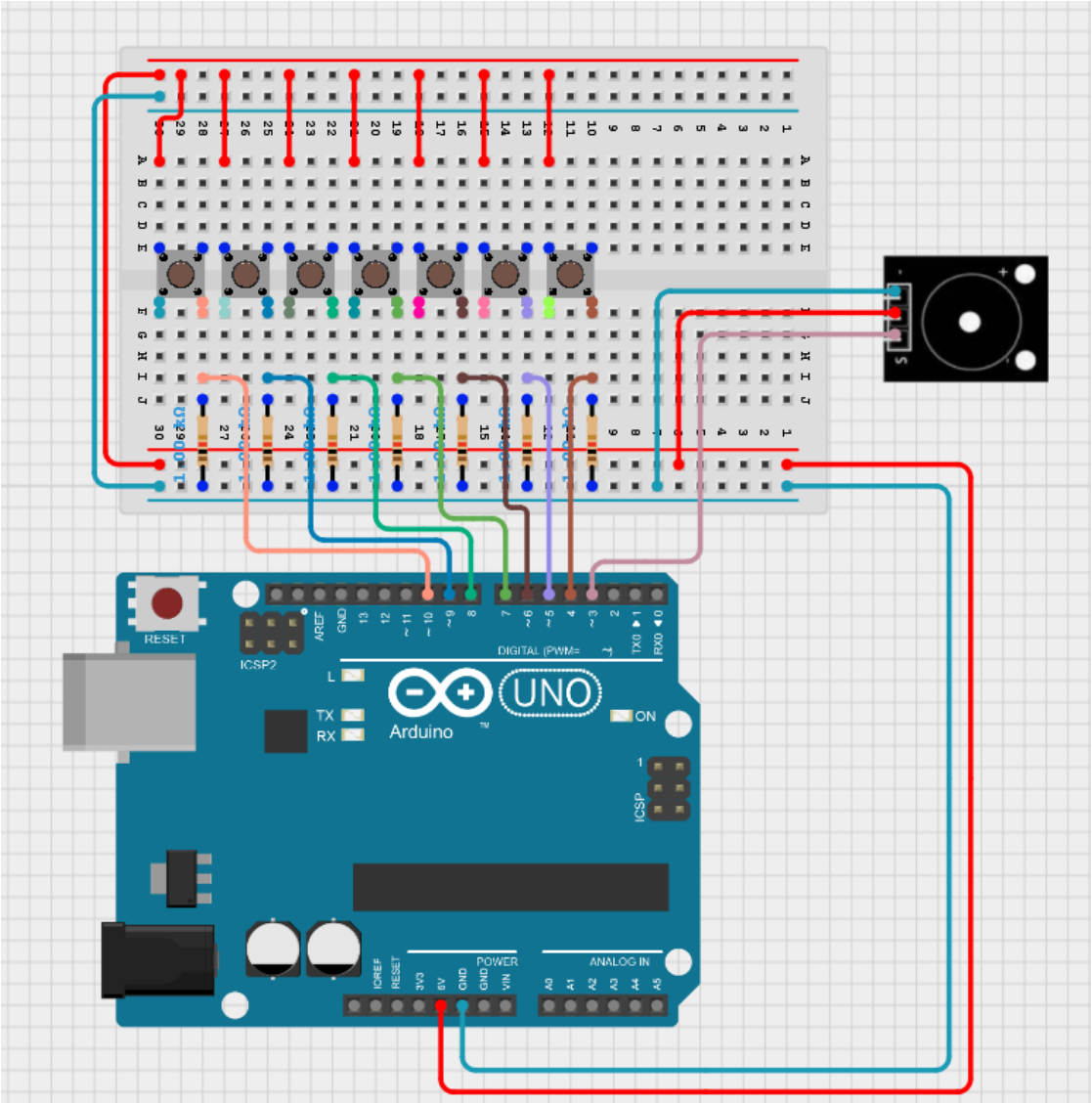
Section 2
Required Components

Here's a list of what you'll need:

- 7 pushbuttons
- 7 resistors 1kΩ
- 1 passive buzzer
- 1 breadboard
- 1 Arduino Uno
- 1 pack of wires

Section 3

Circuit Assembly



https://app.cirkitdesigner.com/project/24717f9f-0f83-4140-a6fe-b03896dc8d5f

Please refer to the image above for the complete circuit wiring. A link to the online schematic can be found under the image in case you need higher resolution.

If this is your first time using a breadboard (the white rectangle in the image with many holes), please refer to our *Breadboard Guide*.

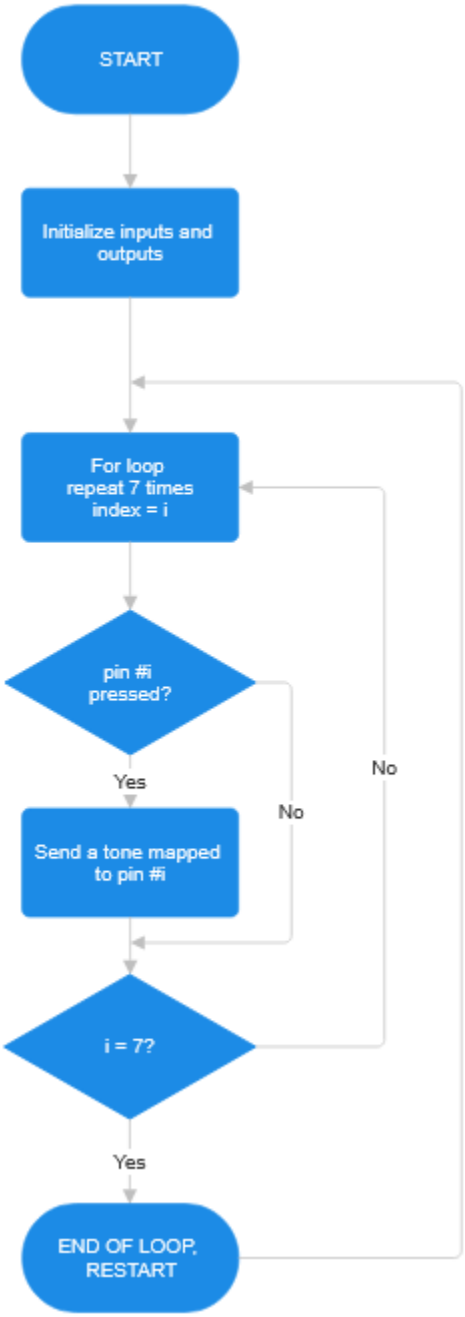
1. **Start by connecting your 2 power bus lines together**
   The two + lines should be connected with a wire and vice-versa for the two - lines.

2. **Place your components as indicated by the diagram**
   For all pushbuttons, make sure they are across the middle divot. The resistors should have one leg connected to the - line and the other connected to one of the button pins. The passive buzzer is oriented weirdly on the diagram, so just make sure the three pins are on separate rows (refer to coding dictionary for reference).

3. **Start by wiring the power connections**
   Connect the Arduino 5V pin to the + line and one of the Arduino GND pins to the - line. For the buttons, make sure the pin that you connect to the + line is horizontally opposite to the pin that the resistor is connected to. Connect the middle pin of the passive buzzer to the + line and the - pin to the - line.

4. **Wire up your signal connections to the Arduino**
   Connect Arduino pin 4 to the row where the button and the resistor meet, at the far right button. Repeat this process for pins 5-10, connecting the pin to its respective button. It's important that the pins are properly mapped to the buttons, as this will make coding a lot easier. Connect pin 3 to the S pin of the passive buzzer.

Section 4
Coding



The code will start with initializing your inputs and outputs. What this means is, you tell the Arduino if the pin you use is an input or an output. You can perform this with the pinMode() function, whose syntax is as such:

```
pinMode([pin number], {OUTPUT/INPUT})
```

e.g. `pinMode(10,INPUT);`

Then, you enter a for loop. This is essentially a loop where you control the number of times you execute a loop. You can use that kind of loop to iterate through elements of a list, or in our case, iterate through a specific set of pins. You can use an index variable "i" to keep track of the number of loops and go through every element. It's important to remember that the index variable is very flexible. It doesn't need to just go from 0 to 7. You can also make the index variable go from 4 to 10, for example. The syntax is as such:

```
for([index is initialized]; [set limit for index]; [set
interval for index])
```

e.g. `for(int i = 4; i <= 10; i++)`

The next part requires you to check for a button press. You can accomplish this with the following function:

```
digitalRead([pin number])
```

e.g. `buttonState = digitalRead(10);`

If you detect a button press, you need to send out a tone with the specific frequency mapped to the button. For the mapping, you can use what's known as a "truth table". The coding dictionary has an entry for this concept, so make sure to read it and ask a volunteer if you have any questions.

For the tone generation, you can use the tone() function. It will generate a signal on a certain pin with a frequency you set. One thing to note is that it can generate a tone <u>while code is running</u>. To keep things simple, we want the program to stall when the tone is generated, so make sure you add a delay() function right after the tone() function, with the delay being the same duration as the tone duration. The syntax is as such:

```
tone([pin #], [frequency of signal], [duration in ms])
```

e.g.
```
tone(3,440,250);
delay(250);
```

Note: it's possible you won't need it, but look at the noTone() function in the coding dictionary. It stops the tone being generated. If you find that you cannot control the duration of your note being played, try using that function.

Shown here is a table that maps notes to frequencies. Use this table to associate your button pressed with any note of your choosing!

NOTE FREQUENCY CHART | HEROIC AUDIO

	Octave 0	Octave 1	Octave 2	Octave 3	Octave 4	Octave 5	Octave 6	Octave 7	Octave 8	Octave 9	Octave 10
C	16.35	32.70	65.41	130.81	261.63	523.25	1046.50	2093.00	4186.01	8372.02	16744.04
C#	17.32	34.65	69.30	138.59	277.18	554.37	1108.73	2217.46	4434.92	8869.84	17739.69
D	18.35	36.71	73.42	146.83	293.66	587.33	1174.66	2349.32	4698.64	9397.27	18794.55
D#	19.45	38.89	77.78	155.56	311.13	622.25	1244.51	2489.02	4978.03	9956.06	19912.13
E	20.60	41.20	82.41	164.81	329.63	659.26	1318.51	2637.02	5274.04	10548.08	
F	21.83	43.65	87.31	174.61	349.23	698.46	1396.91	2793.83	5587.65	11175.30	
F#	23.12	46.25	92.50	185.00	369.99	739.99	1479.98	2959.96	5919.91	11839.82	
G	24.50	49.00	98.00	196.00	392.00	783.99	1567.98	3135.96	6271.93	12543.86	
G#	25.96	51.91	103.83	207.65	415.30	830.61	1661.22	3322.44	6644.88	13289.75	
A	27.50	55.00	110.00	220.00	440.00	880.00	1760.00	3520.00	7040.00	14080.00	
A#	29.14	58.27	116.54	233.08	466.16	932.33	1864.66	3729.31	7458.62	14917.24	
B	30.87	61.74	123.47	246.94	493.88	987.77	1975.53	3951.07	7902.13	15804.26	

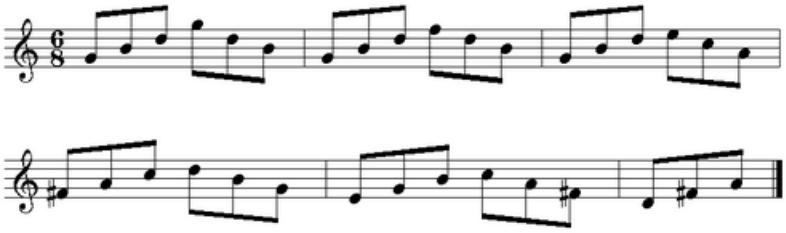
https://producelikeapro.com/blog/note-frequency-chart/

Section 5

Challenge Yourself

1. Arpeggiators

One common feature you will find in many synthesizers is the arpeggiator, which automates an arpeggio. An arpeggio is a type of chord where you play each note of the chord one after the other, instead of all at once with a regular chord. This means that you simply play three notes in a sequence going up or down in pitch. This is how they look on sheet music:



So if arpeggios have always existed, what's the point of arpeggiators? Well, what's neat with automating arpeggios is that we can do it really quickly, and make some neat-sounding stuff. I have attached a YouTube short so you can see what arpeggiators look like on modern equipment:

https://youtube.com/shorts/OL4vIdjklrs?si=vY7zGy_tJLUB36-4

If you have noticed, the synthesizer can't play notes at the same time, which is a hardware limitation. That said, we could do the next best thing and have those notes play one at a time, making some neat effects!

Your objective is this: try creating an arpeggiator (going up or down, your choice) by pressing one of the pushbuttons. You can make multiple, and have them all be controlled by the individual buttons.

2. Pitch wheel (but it's a potentiometer)

Another cool effect you can add to your synth is a pitch wheel. The idea of a pitch wheel is to "slide" or "bend" the pitch of your note, gradually changing the frequency according to a literal wheel on the synthesizer that you could turn back and forth.

This is what they usually look like. I also attached a YouTube short you can watch to really hear what they're supposed to sound like.

https://youtube.com/shorts/9rhWNb-P4vg?si=lsorLUTthKWM3O4X

In lieu of a wheel, we can use a simple potentiometer. You would plug the potentiometer's output to an analog pin and use the analog value to define your pitch bend.

The idea is simple: when the potentiometer goes one way, you increase the frequency, and vice-versa. The farther you go one way, the stronger the effect. Try integrating it into your keyboard, and make it so any button you press can get pitch-bent.